Advanced Cryptography

Elisabeth Oswald (module lead), Sam Collins (teaching assistant)

Adversaries and Security Notions

ROADMAP

- High level view
  - General considerations
  - Types of adversaries
  - Computational power
- Proving techniques
  - Recap: perfectly secret encryption
  - Indistinguishability of ciphertexts
  - Real vs random security
  - Simulation based security considering multiple parties

SECURE MPC

Input privacy

We consider the setting where multiple parties (or players) $P_1, \ldots, P_n$ with inputs $X_1, \ldots, X_n$ wish to jointly compute some function $F(X_1, \ldots, X_n)$, such that **only the function output z is learned** by all parties **but nothing else** is learned about the $X_1, \ldots X_n$ (via the messages that are exchanged). This property is sometimes called **input privacy**.

If only two parties participate, and they e.g. compute the maximum of two values, then the party supplying the smaller value will necessarily learn the input of the other party.

SECURE MPC, CONT.

Robust vs non-robust protocols

Suppose some parties have been corrupted.

If the non-corrupted parties are still guaranteed to produce the correct result we call this **robust** MPC.

If the non-corrupted parties simply abort the protocol and output nothing we call this MPC **with abort**.

If non-corrupted parties output a wrong result, and cannot tell, then this is problematic. When realising MPC with secret sharing, practical protocols often try to do “verifiable secret sharing”.

ROADMAP

- High level view
  - General considerations
  - Types of adversaries
  - Computational power
- Proving techniques
  - Recap: perfectly secret encryption
  - Indistinguishability of ciphertexts
  - Real vs random security
  - Simulation based security considering multiple parties

What is an adversary?

An **adversary** is some entity who wishes to "break" whatever security property we are trying to achieve with or without some help using certain tactics.

To precisely characterise adversaries, we need to precisely define their "security goal", what "help" they may find and/or "tactics" they use in practice.

For instance: an adversary who attacks an MPC protocol where multiple servers interact may be able to run their own code on one (or several) of the servers that allows them to see everything that a server does; an adversary who attacks a bank card may be able to record the EM field of the card whilst it encrypts something.

Active vs passive adversaries

An adversary that can observe communications and side channels of a system is called **passive**. An adversary who can interact with a system and influence its behaviour is called **active**.

What is an adversary, cont.?

Now we bring the notions of active/passive and together with corrupting parties.

Semi-honest vs malicious adversaries

Semi-honest adversary: The corrupted parties co-operate to gather information such as internal states as well as messages that are exchanged during the protocol. The corrupted parties do not deviate from the protocol flow, i.e. they continue to send messages are required by the protocol. The goal is typically to learn the private data of the remaining parties.

Malicious adversary: The corrupted parties co-operate to gather information such as internal states as well as messages. The corrupted parties can deviate from the protocol flow. The goal is typically to subvert the computation of the function.

ROADMAP

- High level view
  - General considerations
  - Types of adversaries
  - Computational power
- Proving techniques
  - Recap: perfectly secret encryption
  - Indistinguishability of ciphertexts
  - Real vs random security
  - Simulation based security considering multiple parties

What is an adversary, cont.?

So far we have looked at what kind of "tactics" an adversary could pursue in the context of MPC, in order to **break** the "goal" of MPC, which is, simply speaking, to keep the inputs of the parties private. How can something "break" in theory/practice?

Security notions

Perfect security. The adversary has unlimited computational powers and yet they cannot break the security notion.

Statistical security. The adversary has unlimited computational powers and they can only break the security notion with an extremely small probability.

Computational security. The adversary has limited computational powers.

One can prove various results for MPC when considering corrupted parties.

ROADMAP

High level view
- General considerations
- Types of adversaries
- Computational power

Proving techniques
- Recap: perfectly secret encryption
- Indistinguishability of ciphertexts
- Real vs random security
- Simulation based security considering multiple parties

What is going on now?

Proving security of secret sharing schemes works, in some ways, similar to arguing about the secrecy of encryption. Thus, it makes sense to revisit, how the security of symmetric encryption is conceptually defined, and how proofs actually work.

To do this, I would like you to:

1. Write down your best understanding of how the security notion for symmetric encryption looks like (don't search it up, go with your memory/gut/instinct).
2. If you want, check with your neighbour(s) how your definitions differ.
3. Discuss: would an algorithms like AES satisfy your security notion?

You have 10-15 minutes for these three tasks. I will then ask for volunteers to share what they came up with.

SYMMETRIC ENCRYPTION

A symmetric encryption $\mathcal{E}$ scheme is given by three algorithms: *Gen*, *Enc*, and *Dec*.

- $Gen : \mathcal{K} \xrightarrow{\$} sk$ generates a secret key via randomly sampling an element from the keyspace
- $Enc : \mathcal{M} \times \mathcal{K} \to \mathcal{C}$ maps an element from the plaintext space to the ciphertext space (using an element from the key space)
- $Dec : \mathcal{C} \times \mathcal{K} \to \mathcal{M}$ recovers the plaintext from the ciphertext (using the secret key)

A symmetric encryption scheme is said to be correct if:
$\forall m \in \mathcal{M}$ and $\forall sk \in \mathcal{K} : Dec_{sk}(Enc_{sk}(m)) = m$. Thus encryption must be reversible.

GOALS FOR ADVERSARIES BREAKING SYMMETRIC ENCRYPTION

- Key recovery: once an adversary has the secret key they can read all messages that were ever encrypted with that key.
- Plaintext recovery: perhaps the adversary has observed ciphertexts and wants to recover the associated plaintexts.

Clearly, key recovery implies plaintext recovery. But the opposite might not hold.

The ultimate goal of any symmetric encryption scheme is to protect the plaintext message.

Defining security for symmetric encryption: formalise the idea that an adversary "does not learn anything about a message upon seeing its ciphertext". This dates back to Shannon.

PERFECTLY SECRET SYMMETRIC ENCRYPTION

Perfect secrecy

An encryption scheme Gen, Enc, Dec with message space $\mathcal{M}$ is **perfectly secret** if for every probability distribution over $\mathcal{M}$, every message $m \in \mathcal{M}$, and every ciphertext $c \in \mathcal{C}$ for which $\Pr[C = c] > 0$:

$$\Pr[M = m | C = c] = \Pr[M = m].$$

Decoded the maths as follows: the left hand of the equation represents what the attacker knows after seeing a ciphertext; the right hand side represents what the attacker knows without seeing the ciphertext.

If both sides are exactly equal, then the ciphertext gives no additional information about the message.

One-time pad encryption

One-time pad (OTP)

The encryption scheme OTP has a message space $\mathcal{M}$, a ciphertext space $\mathcal{C}$ and a key space $\mathcal{K}$ with $|\mathcal{K}| = |\mathcal{C}| = |\mathcal{M}| = 0, 1^n$. *Gen* samples k uniformly at random from $\mathcal{K}$ for a one time use. We define $c = Enc_k(m) = m + k \bmod 2$ (i.e. bit-wise XOR), and decryption likewise to be $m = Dec_k(c) = c + k \bmod 2$.

Thus whenever we use the OTP to encrypt a message, a new key must be sampled randomly from the key space. This key must be of equal length to the message.

One can prove that the one-time pad is perfectly secret by writing down explicitly the probability distributions resulting from the operations of the one-time pad encryption (you can look at this in the background notes).

ROADMAP

High level view
- General considerations
- Types of adversaries
- Computational power

Proving techniques
- Recap: perfectly secret encryption
- Indistinguishability of ciphertexts
- Real vs random security
- Simulation based security considering multiple parties

PERFECT INDISTINGUISHABILITY

One of the major intellectual advances in modern cryptography was to provide an alternative definition for "secrecy", which is best described pictorially.

The adversary is tasked to distinguish between the encryption of two messages of their choice.

If they can win this game, i.e. determine b' such that $b' = b$, the scheme is broken, otherwise it is secure.

The adversary has unlimited power.

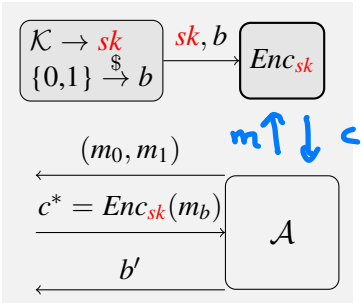

Figure: Exp ind-pas. Win if $b' = b$.

PERFECT INDISTINGUISHABILITY, CONT.

In practice, adversaries do get help, leading to notions such as IND-CPA (CPA = chosen plaintext attack) and IND-CCA (CCA=chosen ciphertext attack), but they are not infinitely powerful.

Can a modern block cipher satisfy IND-PAS or IND-CPA in a computational setting?

ILLUSTRATION BASED ON THE ONE-TIME PAD

These “intuitions” and “mental models” for proofs may sound “easy” but translating them into a concrete, reasonably formal proof on paper is not easy.

It took the crypto community decades to get to a point where many cryptographer’s can do this correctly, and teaching proofs as part of undergraduate curricula is still not the norm.

We are going to have a go at understanding how a proof based on indistinguishability of ciphertext works.

INDISTINGUISHABILITY OF CIPHERTEXTS: REDRAW GAME

To produce c either the left or the right oracle is used. The adversary wins if they can tell if c comes from the left or the right oracle: i.e. if there are values that are more likely in the left world, then in the right world.

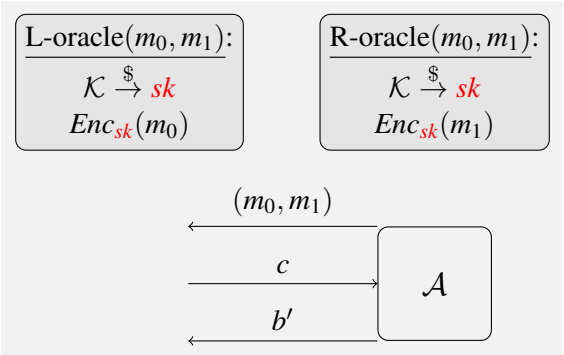

Figure: Exp LR-Pas. Win if $b' = b$.

INDISTINGUISHABILITY OF CIPHERTEXTS: OTP

Drop in the specific encryption function of the one-time pad (OTP).

Proof argument: $\mathcal{K} \xrightarrow{\$} sk$ delivers a key sk with $\Pr[k] = 1/|\mathcal{K}|$. For fixed $m \in \mathcal{M}$, the mapping $c = m \oplus k$ is bijective: i.e. each c is equally likely.

The c returned from the left oracle, and the c returned from the right oracle are both uniformly distributed; aka they cannot be distinguished.

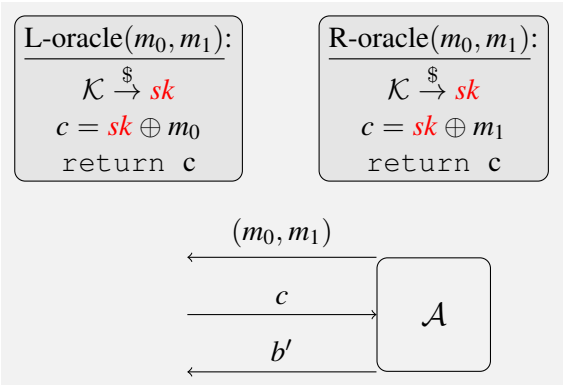

Figure: Exp LR-Pas. Win if $b' = b$.

Left (m_L, m_R) :

$k \xleftarrow{\$} \{0,1\}^\ell$

$c = k \oplus m_L$

return (k, c)

Right (m_L, m_R) :

$k \xleftarrow{\$} \{0,1\}^\ell$

$c = k \oplus m_R$

return (k, c)

$m_L = 0 \ldots 0$

$m_R = 1 \ldots 1$

L : $c = k \oplus m_L$

$c = k$

(k, k)

R : $c = k \oplus m_R$

$c = \bar{k}$

$(k, \bar{k})$

Left (m)

$x \xrightarrow{\$} 20_{10}{}^{e}$

$y = x \oplus m$

return (x, y)

Right (m)

$y \xrightarrow{\$} 20_{10}{}^{e}$

$x = y \oplus m$

return (x, y)

1) Define Two-Time Pad Encrypt. Scheme.

$(m_1, m_2) \rightarrow (c_1, c_2)$

k = same size as m_i

2) Write Left oracle and write a Right Oracle

3) Are they distinguishable?

Enc(m):

$k \xleftarrow{\$} \{0,1\}^e$

$c_1 = m_1 \oplus k$

$c_2 = m_2 \oplus k$

return $c = c_1 \| c_2$

$m = m_1 \| m_2$

$m_L = 0 \ldots 0 \| 0 \ldots 0$

$m_R = 0 \ldots 0 \| 1 \ldots 1$

ROADMAP

High level view
- General considerations
- Types of adversaries
- Computational power

Proving techniques
- Recap: perfectly secret encryption
- Indistinguishability of ciphertexts
- Real vs random security
- Simulation based security considering multiple parties

TOWARDS SIMULATION BASED SECURITY

I am trying to get you to understand the "mental models" that underpin how we understand the intuition of "not learning anything" or there being "no information about".

Besides the notion of indistinguishable encryptions (sometimes also referred to as left-right notion), another idea was formalised, which is as follows:

- we have adversaries $\mathcal{A}_r$ who get as information a ciphertext, and the length of the underlying plaintext;
- we have adversaries $\mathcal{A}_i$ who get as information only the length of the underlying plaintext;

we consider a system as secure if $\mathcal{A}_r$ and $\mathcal{A}_i$ learn "about the same amount of information".

Ergo: the ciphertext contributes nothing to $\mathcal{A}_r$.

Real vs. ideal world adversaries

$\mathcal{A}_r$ represents the situation in the real world.

$\mathcal{A}_i$ represents the situation in a world that is ideal from a cryptographer's perspective: no information at all about the message is ever passed on to the adversary.

If both adversaries are equally good, then the availability of the ciphertext in the real world does not give the adversary any information.

TRANSLATING THIS INTO A PROVE TECHNIQUE/GAME

Showing that the real and ideal world are the same (or demonstrate a problem by showing that they are not) requires another idea.

This is done based on defining a simulator that is capable of simulating the ideal world in such a way that an adversary cannot tell whether they are given "stuff" that comes from a protocol run in the real world, or whether they are given "stuff" that comes from a protocol run in the ideal world.

REAL VS RANDOM SECURITY: OTP

The next step is the mental model behind simulation based security.

Distinguish between result of an oracle that resembles the "real world" vs an oracle that resembles the "ideal" world.

In the "ideal world" there is no key, and return values are random values (hence "random" world.).

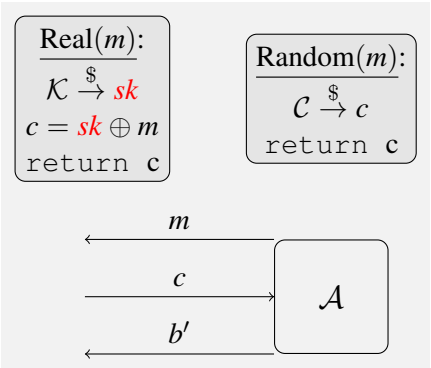

Figure: Exp RR-Pas. Win if $b' = b$.

REAL VS RANDOM SECURITY: OTP

Distribution of c in real world: using the same argument as before, we conclude that $\Pr[c] = 1/|\mathcal{C}|$.

Distribution of c in the random world: $\Pr[c] = 1/|\mathcal{C}|$.

Therefore OTP offers one-time real-vs-random security.

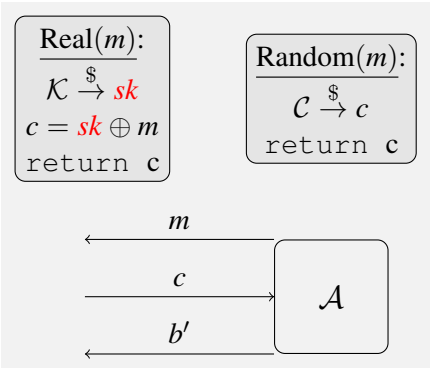

Figure: Exp RR-Pas. Win if $b' = b$.

Relationships between mental models/security notions

It would be tempting to think that, because these notions look similar, they are implying each other. However, this is not the case.

If one can prove security (for some scheme) in the real-random sense, then security in the left-right sense follows immediately.

But it is not true that left-right security implies real-random security.

Relationships between mental models/security notions

Disproving a statement only requires a single counterexample.

Let us disprove: Left-right security implies real-random security.

Consider and encryption scheme that appends a fixed sequence to any ciphertext:
`Enc`: $\mathcal{E}_k(m)||$`Str`, $\mathcal{E}$ some secure encryption, and `Str` some fixed string.

Even if $\mathcal{E}$ would be real-random secure, `Enc` cannot be although it is left-right secure.

This is because in the left-right setting the fixed string has not impact (it occurs in both oracle), but in the real-random world it implies that the ciphertext can be distinguished with very high probability from a random string.

So where is that simulator?

In the example that is based on the OTP, the simulator is basically how we construction the “random” oracle.

In the case of the simple OTP, the simulator is not particularly interesting, and it is pretty obvious how it should look like.

If we try to do this for more complex schemes or protocols where parties can interact, then writing the simulator is non-trivial.

ROADMAP

High level view
- General considerations
- Types of adversaries
- Computational power

Proving techniques
- Recap: perfectly secret encryption
- Indistinguishability of ciphertexts
- Real vs random security
- Simulation based security considering multiple parties

LEAKING CIRCUITS

This is now an example that "looks ahead" in the sense that we will get to a real world version when looking at side channel countermeasures.

Suppose that we have a secret bit x that is distributed across three parties P_i and each party has an x_i such that $x = x_1 \oplus x_2 \oplus x_3$. The "shares" x_i are constructed as follows: x_1 and x_2 are sampled uniformly at random, and $x_3 = x \oplus x_1 \oplus x_2$.

The parties compute $r = (x_1 \cdot x_2) \oplus x_3$.

We want to understand the security implication if one or multiple parties were corrupted (semi-honest security).

LEAKING CIRCUITS, CONT.

Assume leakage on shares only. P_i **corrupted**: any x_1 and x_2 are drawn from a uniform distribution, thus the simulator for the random world just samples a value from the uniform distribution. If x_3 is leaked, then observe that $x_3 = s \oplus x_1 \oplus x_2$, thus x_3 is the one-time pad encryption of s. We know that this cannot be distinguished from drawing x_3 uniformly, hence the simulator can produce x_3 in the same way as it produces x_1 and x_2.

Suppose that there was also leakage on $x_1 \cdot x_2$. P_1 **or** P_2 **corrupted:** If the product of x_1 and x_2 is known and one of the two values, then the simulator has a bit more to do. This is because if $x_1 \cdot x_2$ is, e.g. 0 and $x_1 = 1$, then $\Pr[x_2 = 0] = 1$. Thus now the simulator must actually know=choose x_2 in order to produce a perfect simulation (i.e. ensure that the distribution of values in the random world cannot be distinguished from the distribution of values from the real world).